第一次作业

并行程序设计与算法（理论）

元朗曦

23336294

题 1.

1. 结合存储组织方式，说明 **distributed-memory system** 与 **shared-memory system** 的区别，并解释为什么在 distributed-memory system 中，处理器之间不能像 shared-memory system 那样直接访问同一块主存数据。
2. 什么是 **NUMA（Non-Uniform Memory Access）**？它属于 shared-memory system 还是 distributed-memory system？请说明理由。

解.

1. 两者的主要区别在存储组织与地址空间：**distributed-memory system** 中各处理器（节点）拥有独立本地内存，地址空间彼此分离，数据交换依赖消息传递；**shared-memory system** 中多个处理器共享同一全局地址空间，可直接读写同一主存。

   distributed-memory system 不能像 shared-memory system 那样直接访问同一块主存，是因为其内存在硬件上分散、地址空间不统一，远端内存地址对本地处理器不可直接寻址，因此跨节点访问必须通过通信机制完成，而非本地 load/store 指令。

2. **NUMA（Non-Uniform Memory Access，非一致内存访问）**指在统一共享地址空间下，不同内存位置的访问代价不同：访问本地内存更快，访问远端内存更慢。

   NUMA 属于 shared-memory system。因为处理器仍可直接访问全局内存，编程语义仍是共享内存；其“非一致”体现在访问延迟差异，而不是必须通过消息传递访问他处数据。

题 2.

1. 解释 process、thread 和 multitasking 的含义。
2. 说明 process 与 thread 的一个重要区别。
3. 为什么 thread 常被称为“轻量级”的执行单元？

解.

1. **进程（process）**是操作系统分配资源的基本单位，拥有独立的地址空间、代码、数据和系统资源；**线程（thread）**是进程内的执行单元，多个线程共享同一进程的地址空间和资源；**多任务（multitasking）**是操作系统同时管理多个进程或线程执行的能力。
2. 进程与线程的一个重要区别在于资源隔离：进程之间相互独立，无法直接访问对方内存；而线程共享同一进程内存，可以直接读写共享数据。
3. 线程常被称为“轻量级”的执行单元，因为它们共享进程资源，创建和切换线程的开销较小；相比之下，创建和切换进程涉及更多资源分配和上下文切换，因此更“重量级”。

题 3. 阅读下面的 Pthreads 代码片段，回答问题：

```
for (thread = 0; thread < thread_count; thread++)
  pthread_create(&thread_handles[thread], NULL, Hello, (void*)thread);
```

```
for (thread = 0; thread < thread_count; thread++)
  pthread_join(thread_handles[thread], NULL);
```

1. `pthread_create` 的作用是什么?
2. `pthread_join` 的作用是什么?
3. 对于默认创建的 joinable 线程，如果主线程既不 `detach` 它们，也不对它们调用 `pthread_join`，程序能否“正确结束”? 请结合线程资源回收和进程终止行为说明原因。

解.

1. `pthread_create` 的作用是创建一个新线程，并使其从指定的线程函数（此处为 `Hello`）开始执行；同时将该线程的标识符写入 `thread_handles[thread]`，便于后续管理（如 `join`）。
2. `pthread_join` 的作用是阻塞调用者（通常为主线程），直到目标线程结束；对于 `joinable` 线程，它还负责完成线程终止后的资源回收，并可获取线程返回值（此处传 `NULL` 表示不接收返回值）。
3. 对默认创建的 `joinable` 线程，若主线程既不 `detach` 也不 `pthread_join`：线程函数即使执行完，其线程控制块等资源也不会被及时回收，会处于“已终止但未回收”的状态（可类比“僵尸线程”）；若此类线程持续累积，可能造成资源泄漏。

   从进程终止角度看，若主线程从 `main` 返回或调用 `exit`，进程会整体结束，其他线程也会被终止，操作系统最终会回收进程资源；但这不属于规范的线程回收方式。因此，这种写法通常不应视为“正确结束”，更合理的做法是对线程执行 `pthread_join` 或将其设为 detached。

题 4. 下面是课件中关于多线程字符串分词的例子背景：多个线程同时调用 `strtok` 处理不同输入行时，程序可能产生错误结果。

1. 什么是 **thread-safe（线程安全）**？
2. 为什么 `strtok` 不是线程安全的?
3. 课件中给出的线程安全替代函数是什么?

解.

1. 线程安全是指：当多个线程并发调用同一函数（或访问同一对象）时，在不额外引入错误同步的前提下，函数仍能保持语义正确，不会因竞争条件产生未定义或错误结果。
2. `strtok` 不是线程安全的，主要因为它在函数内部使用静态状态记录“上一次分词位置”。多个线程并发调用时会共享并相互覆盖这份状态，导致分词过程互相干扰，从而产生错误结果。
3. 课件中给出的线程安全替代函数是 `strtok_r`（re-entrant 版本）。它将上下文状态通过调用者提供的变量保存，不依赖共享的静态内部状态，因此可安全用于多线程场景。

题 5. 某程序总共执行 1.2×10^{11} 条指令。现比较它在两种处理器配置下的执行情况：

- 配置 A：主频为 2.4 GHz，平均 CPI = 2.0。
- 配置 B：主频为 3.0 GHz，平均 CPI = 1.2。

请计算：

1. 在配置 A 下的执行时间。
2. 在配置 B 下的执行时间。
3. 配置 B 相对于配置 A 的加速比。
4. 若仅把主频从 2.4 GHz 提高到 3.0 GHz，但 CPI 仍保持 2.0，则执行时间是多少？并据此说明：在这个例子里，“仅提高主频”和“提高主频同时降低 CPI”相比，哪一种方案更有效。

解.

1. 配置 A 下的执行时间：

$$T_A = \frac{1.2 \times 10^{11} \times 2.0}{2.4 \times 10^9} = 100 \text{ s}.$$

2. 配置 B 下的执行时间：

$$T_B = \frac{1.2 \times 10^{11} \times 1.2}{3.0 \times 10^9} = 48 \text{ s}.$$

3. 加速比：

$$S = \frac{T_A}{T_B} = \frac{100}{48} \approx 2.08.$$

4. 若仅把主频从 2.4 GHz 提高到 3.0 GHz，CPI 仍为 2.0：

$$T = \frac{1.2 \times 10^{11} \times 2.0}{3.0 \times 10^9} = 80 \text{ s}.$$

   对比可得：仅提高主频时，执行时间由 100 s 降到 80 s（加速比 $S = \frac{100}{80} = 1.25$）；提高主频同时降低 CPI 时，执行时间降到 48 s（加速比约 2.08）。因此在该例中，“提高主频同时降低 CPI”明显更有效。

题 6. 在 `MPI_COMM_WORLD` 中共有 4 个进程。设进程 1、2、3 分别执行以下发送操作：

- 进程 1：`MPI_Send(&u, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);`，其中 `u = 10`。
- 进程 2：`MPI_Send(&v, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);`，其中 `v = 20`。
- 进程 3：`MPI_Send(&w, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);`，其中 `w = 30`。

进程 0 依次执行以下接收操作：

- `R1: MPI_Recv(&a, 1, MPI_INT, 3, 0, MPI_COMM_WORLD, &s1);`

- `R2: MPI_Recv(&b, 1, MPI_INT, 2, 1, MPI_COMM_WORLD, &s2);`
- `R3: MPI_Recv(&c, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &s3);`

请回答：

1. 变量 `a`、`b`、`c` 的值分别是多少。
2. `s1.MPI_SOURCE`、`s1.MPI_TAG` 的值分别是多少？`s2` 和 `s3` 同理。
3. 如果把 `R2` 中的 `tag` 误写成 `0`，那么这三条接收语句中哪一条会无法匹配？请说明原因。

解.

1. 由接收端给定的 `(source, tag)` 可知：
   - `R1` 只会匹配来自进程 3、`tag == 0` 的消息，因此 `a = 30`。
   - `R2` 只会匹配来自进程 2、`tag == 1` 的消息，因此 `b = 20`。
   - `R3` 只会匹配来自进程 1、`tag == 0` 的消息，因此 `c = 10`。
2. `MPI_Status` 中记录的是“实际匹配到的消息”的源和标记，因此：
   - `s1.MPI_SOURCE = 3`，`s1.MPI_TAG = 0`；
   - `s2.MPI_SOURCE = 2`，`s2.MPI_TAG = 1`；
   - `s3.MPI_SOURCE = 1`，`s3.MPI_TAG = 0`。
3. 若把 `R2` 的 `tag` 误写为 `0`，则无法匹配的是 `R2`。原因是进程 2 发送的消息标记是 `tag == 1`，而 `R2` 要求 `source == 2 && tag == 0`，系统中不存在这样的消息，因此 `R2` 会阻塞等待；由于接收按顺序执行，`R3` 也不会被执行到。